MySQL Cheatsheet

Complete Reference Guide

1. SQL Command Categories

DDL - Data Definition Language

Used to define and modify database structure.

Commands: CREATE, ALTER, DROP, TRUNCATE, RENAME

DML - Data Manipulation Language

Used to manipulate data within tables.

Commands: INSERT, UPDATE, DELETE, SELECT

DCL - Data Control Language

Used to control access to data in database.

Commands: GRANT, REVOKE

TCL - Transaction Control Language

Used to manage transactions in the database.

Commands: COMMIT, ROLLBACK, SAVEPOINT, SET TRANSACTION

DQL - Data Query Language

Used to query data from the database.

Commands: SELECT

2. DDL Commands

CREATE DATABASE

```
CREATE DATABASE database_name;
CREATE DATABASE IF NOT EXISTS database_name;
```

USE DATABASE

```
USE database_name;
```

DROP DATABASE

```
DROP DATABASE database_name;
DROP DATABASE IF EXISTS database_name;
```

CREATE TABLE

```
CREATE TABLE table_name ( column1 datatype constraint, column2 datatype constraint, PRIMARY KEY (column1) );
```

Example:

```
CREATE TABLE students ( id INT AUTO_INCREMENT, name VARCHAR(100) NOT NULL, email VARCHAR(100) UNIQUE, age INT, PRIMARY KEY (id) );
```

ALTER TABLE

```
ALTER TABLE table_name ADD column_name datatype;
ALTER TABLE table_name DROP COLUMN column_name;
ALTER TABLE table_name MODIFY COLUMN column_name datatype;
ALTER TABLE table_name RENAME TO new_table_name;
```

DROP TABLE

```
DROP TABLE table_name;
DROP TABLE IF EXISTS table_name;
```

TRUNCATE TABLE

```
TRUNCATE TABLE table_name;
```

(Removes all rows but keeps table structure)

3. DML Commands - CRUD Operations

INSERT (Create)

```
INSERT INTO table_name (col1, col2) VALUES (val1, val2);
INSERT INTO table_name VALUES (val1, val2, val3);
INSERT INTO table_name (col1, col2) VALUES
(val1, val2), (val3, val4);
```

Example:

```
INSERT INTO students (name, email, age) VALUES ('John Doe', 'john@email.com', 20);
```

SELECT (Read)

```
SELECT * FROM table_name;
SELECT col1, col2 FROM table_name;
SELECT DISTINCT column FROM table_name;
SELECT * FROM table_name WHERE condition;
```

Example:

```
SELECT name, email FROM students WHERE age > 18;
```

UPDATE (Update)

```
UPDATE table_name SET col1 = val1 WHERE condition;
UPDATE table_name SET col1 = val1, col2 = val2 WHERE condition;
```

Example:

```
UPDATE students SET age = 21 WHERE name = 'John Doe';
```

DELETE (Delete)

```
DELETE FROM table_name WHERE condition;
DELETE FROM table_name; (Deletes all rows)
```

Example:

```
DELETE FROM students WHERE age < 18;
```

4. WHERE Clause & Operators

Comparison Operators

```
=, !=, <>, >, <, >=, <=
```

Logical Operators

```
AND, OR, NOT
SELECT * FROM students WHERE age > 18 AND age < 25;
```

BETWEEN

```
SELECT * FROM students WHERE age BETWEEN 18 AND 25;
```

IN

```
SELECT * FROM students WHERE age IN (18, 20, 22);
```

LIKE - Pattern Matching

% = wildcard for any number of characters

_ = wildcard for exactly one character

```
SELECT * FROM students WHERE name LIKE 'J%';
SELECT * FROM students WHERE name LIKE '%ohn%';
SELECT * FROM students WHERE name LIKE 'J___';
```

IS NULL / IS NOT NULL

```
SELECT * FROM students WHERE email IS NULL;
SELECT * FROM students WHERE email IS NOT NULL;
```

5. Sorting, Grouping & Limiting

ORDER BY - Sorting Results

```
SELECT * FROM students ORDER BY age ASC;
SELECT * FROM students ORDER BY age DESC;
SELECT * FROM students ORDER BY age DESC, name ASC;
```

LIMIT - Limiting Results

```
SELECT * FROM students LIMIT 10;
SELECT * FROM students LIMIT 5 OFFSET 10;
SELECT * FROM students LIMIT 10, 5; (offset, count)
```

GROUP BY - Grouping Data

```
SELECT column, COUNT(*) FROM table GROUP BY column;
```

Example:

```
SELECT age, COUNT(*) FROM students GROUP BY age;
```

HAVING - Filter Grouped Results

(WHERE filters rows before grouping, HAVING filters after)

```
SELECT age, COUNT(*) as count FROM students GROUP BY age HAVING count > 5;
```

Aggregate Functions

```
COUNT(*), COUNT(column), SUM(column), AVG(column)
MIN(column), MAX(column)
```

Example:

```
SELECT AVG(age) FROM students;
SELECT COUNT(*) FROM students WHERE age > 18;
SELECT department, AVG(salary) as avg_salary FROM employees GROUP BY department HAVING
avg_salary > 50000;
```

6. JOINS - Combining Tables

INNER JOIN

Returns records with matching values in both tables

```
SELECT students.name, courses.course_name FROM students INNER JOIN enrollments ON students.id
= enrollments.student_id INNER JOIN courses ON enrollments.course_id = courses.id;
```

LEFT JOIN (LEFT OUTER JOIN)

Returns all records from left table, matching from right

```
SELECT students.name, enrollments.course_id FROM students LEFT JOIN enrollments ON students.id
= enrollments.student_id;
```

RIGHT JOIN (RIGHT OUTER JOIN)

Returns all records from right table, matching from left

```
SELECT students.name, courses.course_name FROM students RIGHT JOIN courses ON students.major =
courses.department;
```

CROSS JOIN

Returns Cartesian product of both tables

```
SELECT students.name, courses.course_name FROM students CROSS JOIN courses;
```

SELF JOIN

Join table with itself

```
SELECT e1.name as Employee, e2.name as Manager FROM employees e1 JOIN employees e2 ON
e1.manager_id = e2.id;
```

7. Subqueries (Nested Queries)

Subquery in WHERE Clause

```
SELECT name FROM students WHERE age = (SELECT MAX(age) FROM students);

SELECT name FROM students WHERE id IN (SELECT student_id FROM enrollments WHERE course_id =
101);
```

Subquery in FROM Clause

```
SELECT avg_age FROM (SELECT AVG(age) as avg_age FROM students GROUP BY department) as
dept_avg;
```

Correlated Subquery

```
SELECT name, salary FROM employees e1 WHERE salary > (SELECT AVG(salary) FROM employees e2
WHERE e1.dept = e2.dept);
```

EXISTS

```
SELECT name FROM students s WHERE EXISTS (SELECT 1 FROM enrollments e WHERE e.student_id =
s.id);
```

8. Transaction Control Language (TCL)

START TRANSACTION

```
START TRANSACTION;
BEGIN;
```

COMMIT

Saves all changes made in the transaction

```
COMMIT;
```

ROLLBACK

Undoes all changes made in the transaction

```
ROLLBACK;
```

SAVEPOINT

Creates a point within transaction to rollback to

```
SAVEPOINT savepoint_name;
ROLLBACK TO savepoint_name;
```

Transaction Example:

```
START TRANSACTION; UPDATE accounts SET balance = balance - 100 WHERE id = 1; UPDATE accounts
SET balance = balance + 100 WHERE id = 2; COMMIT;
```

9. Constraints

PRIMARY KEY

Uniquely identifies each record

```
CREATE TABLE users (id INT PRIMARY KEY, name VARCHAR(100));
```

FOREIGN KEY

Links two tables together

```
CREATE TABLE orders ( id INT PRIMARY KEY, customer_id INT, FOREIGN KEY (customer_id)
REFERENCES customers(id) );
```

UNIQUE

Ensures all values in column are different

```
email VARCHAR(100) UNIQUE
```

NOT NULL

Ensures column cannot have NULL value

```
name VARCHAR(100) NOT NULL
```

CHECK

Ensures values meet specific condition

```
age INT CHECK (age >= 18)
```

DEFAULT

Sets default value for column

```
status VARCHAR(20) DEFAULT 'active'
```

AUTO_INCREMENT

Automatically generates unique number

```
id INT AUTO_INCREMENT PRIMARY KEY
```

10. Common Data Types

Numeric Types

```
INT, BIGINT, SMALLINT, TINYINT
DECIMAL(p, s), NUMERIC(p, s)
FLOAT, DOUBLE
```

String Types

```
CHAR(n) - Fixed length
VARCHAR(n) - Variable length
TEXT, MEDIUMTEXT, LONGTEXT
```

Date and Time Types

```
DATE - YYYY-MM-DD
TIME - HH:MM:SS
DATETIME - YYYY-MM-DD HH:MM:SS
TIMESTAMP
YEAR
```

Other Types

```
BOOLEAN (TINYINT(1))
BLOB - Binary Large Object
ENUM('value1', 'value2', ...)
```

11. Useful MySQL Functions

String Functions

```
CONCAT(str1, str2, ...) - Concatenate strings
UPPER(str), LOWER(str) - Change case
LENGTH(str) - String length
SUBSTRING(str, start, length) - Extract substring
TRIM(str), LTRIM(str), RTRIM(str) - Remove spaces
```

Numeric Functions

```
ROUND(number, decimals)
CEIL(number), FLOOR(number)
ABS(number) - Absolute value
POWER(base, exponent)
```

Date Functions

```
NOW() - Current date and time
CURDATE() - Current date
CURTIME() - Current time
DATE_ADD(date, INTERVAL value unit)
DATEDIFF(date1, date2)
YEAR(date), MONTH(date), DAY(date)
```

12. Indexes & Performance

CREATE INDEX

```
CREATE INDEX idx_name ON table_name(column);
CREATE UNIQUE INDEX idx_name ON table_name(column);
```

DROP INDEX

```
DROP INDEX idx_name ON table_name;
```

SHOW INDEXES

```
SHOW INDEXES FROM table_name;
```

13. DCL Commands

GRANT - Give Privileges

```
GRANT SELECT, INSERT ON database.* TO 'user'@'host';
GRANT ALL PRIVILEGES ON *.* TO 'user'@'host';
```

REVOKE - Remove Privileges

```
REVOKE SELECT ON database.* FROM 'user'@'host';
```

14. Useful Commands

```
SHOW DATABASES;
SHOW TABLES;
DESCRIBE table_name; (or DESC table_name;)
SHOW CREATE TABLE table_name;
SHOW COLUMNS FROM table_name;
```

15. SQL Query Execution Order

Understanding the order helps write better queries:

Step	Clause	Description
1	FROM	Choose and join tables
2	WHERE	Filter rows
3	GROUP BY	Group rows
4	HAVING	Filter groups
5	SELECT	Select columns
6	DISTINCT	Remove duplicates
7	ORDER BY	Sort results
8	LIMIT	Limit number of rows

16. Tips & Best Practices

• Always use WHERE clause in UPDATE and DELETE to avoid changing all rows

• Use LIMIT when testing queries on large tables

• Create indexes on frequently queried columns

• Use JOIN instead of subqueries when possible for better performance

• Always backup database before making structural changes

• Use transactions for related operations to maintain data integrity

• Normalize your database to reduce redundancy

• Use appropriate data types to save storage and improve performance

• Use EXPLAIN to analyze query performance

• Avoid SELECT * in production code; specify needed columns